

Week 4 · Lesson 3 — gzip & Compression (REQUIRED for this project)

Time: ~3–4h · **Goal:** Understand what gzip compression is, why the EVB-01 firmware *needs* it, and exactly how to serve pre-compressed files from the ESP32. This lesson is non-negotiable — the UI won't fit on the device without it.

Why this matters for EVB-01

The ESP32-S3 has **4MB of flash total**, and only about **~1.5MB** is available for your web UI (the SPIFFS filesystem in `data/`). A React build can be a few hundred KB of JS/CSS. **gzip shrinks text assets by ~70–80%** (a 600KB JS bundle → ~120–180KB), which is the difference between fitting comfortably and not fitting at all. It also means the device sends *less data over WiFi*, so the dashboard loads faster — and the chip doesn't waste CPU compressing on the fly because you pre-compress at build time.

Core concepts

What gzip actually does

- gzip uses the **DEFLATE** algorithm = **LZ77** (replaces repeated strings with short back-references) + **Huffman coding** (shorter codes for frequent bytes). Source code is full of repetition and whitespace, so it compresses dramatically.
- You compress a file `index.js` → `index.js.gz`. The browser **decompresses it automatically** — if the server tells it to.

How the browser knows to decompress (the HTTP handshake)

- The browser sends a request header: `Accept-Encoding: gzip` ("I can handle gzip").
- The server must respond with: `Content-Encoding: gzip` ("this body is gzipped").
- **If the server forgets that response header, the browser shows gibberish.** This is the #1 gzip bug.

⚠ The ESP32-specific gotcha (memorize this)

On the **ESP32 Arduino core**, `server.serveStatic()` does **NOT** reliably handle `.gz` files:
- It ignores `Accept-Encoding`, and if both `index.html` and `index.html.gz` exist it serves the

uncompressed one.

- (This is different from the ESP8266 core, which *does* auto-serve `.gz`. Tutorials written for ESP8266 will mislead you.)

The reliable technique for EVB-01:

1. Put **only** the `.gz` versions in `data/` (delete the uncompressed originals to save space).
2. Register a **custom handler** instead of relying on `serveStatic`, open the `.gz` from SPIFFS, set the header **manually**, and stream it:

```
cpp server.on("/assets/index.js", HTTP_GET, []() { File f =  
SPIFFS.open("/assets/index.js.gz", "r"); server.setHeader("Content-Encoding", "gzip");  
server.streamFile(f, "application/javascript"); f.close(); });
```

(You'll wire this up properly in Lesson 4 with your manager — firmware edits are a collaboration, not solo work.)

Watch / read (in order)

- **[CORE]** MDN — *gzip compression* (glossary): https://developer.mozilla.org/en-US/docs/Glossary/gzip_compression
- **[CORE]** MDN — *Content-Encoding* header: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Content-Encoding> and *Accept-Encoding*: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Accept-Encoding>
- **[CORE]** Reference issues on the ESP32 `.gz` behavior (so you trust the gotcha above): <https://github.com/espressif/arduino-esp32/issues/5960>
- **[OPTIONAL]** `vite-plugin-compression2` (auto-emits `.gz` at build time): <https://github.com/nonzzz/vite-plugin-compression2>

Practice (do this) → push to git

1. Take your `dist/` from Lesson 2. gzip the JS/CSS manually (e.g. `gzip -9 -k dist/assets/*.js dist/assets/*.css`) and **compare sizes** — record the before/after and the % saved in `notes.md`.
2. Install `vite-plugin-compression2`, add it to `vite.config.js`, rebuild, and confirm `.gz` files now appear automatically in `dist/`.
3. In `notes.md`, write the two HTTP headers involved and the one-sentence ESP32 gotcha in your own words.
4. Commit: `git commit -m "Week 4: gzip compression - sizes compared + vite gzip plugin wired"` and push.

Before you move on, you should be able to...

- ☐ Explain why gzip is required to fit the UI on the device, with a rough % figure.
- ☐ Name the request and response headers that make gzip work.
- ☐ Explain why `serveStatic` alone won't serve your `.gz` files on the ESP32, and what to do instead.